
LAPORAN PRAKTIKUM IOT II



Oleh:

NAMA : ALIZAR PRATAMA

NPM : 25782089

KELAS : TRI – 3D

Dosen:

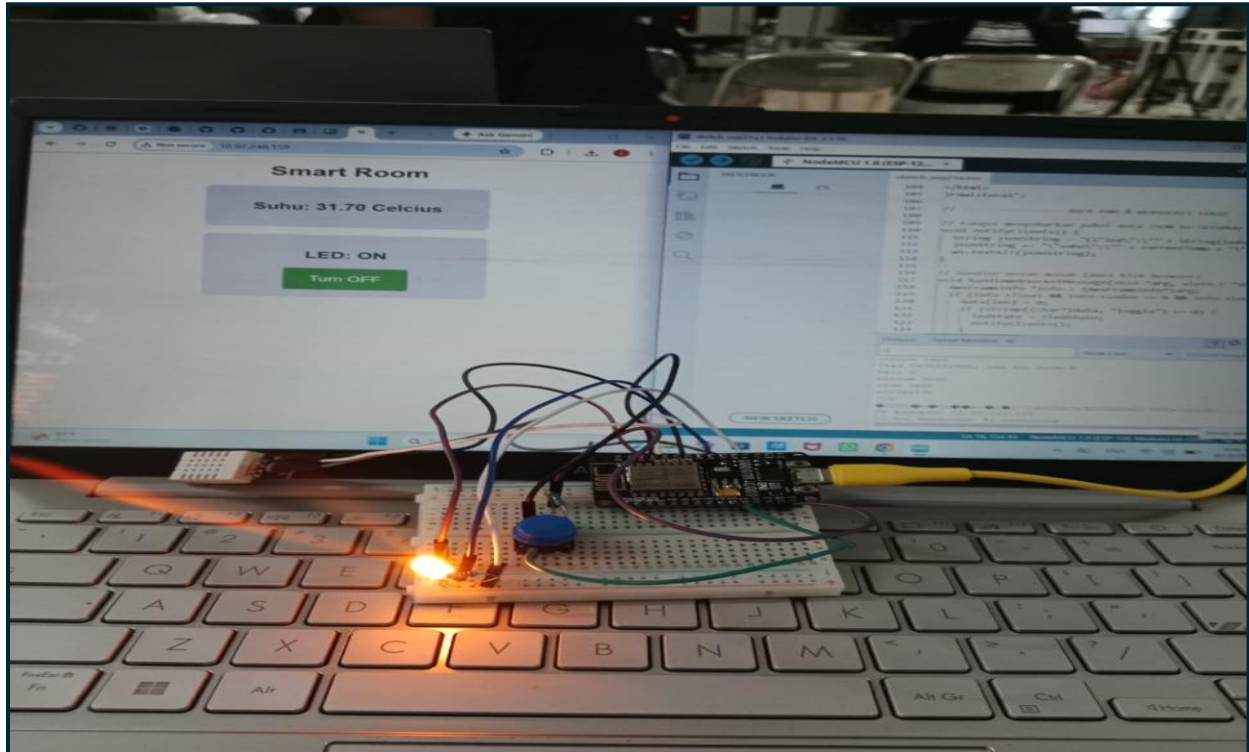
Dr. Ir Septafiansyah Dwi Putra, S.T., MT.

Agus Ambarwari, S.Pd., M.Kom.

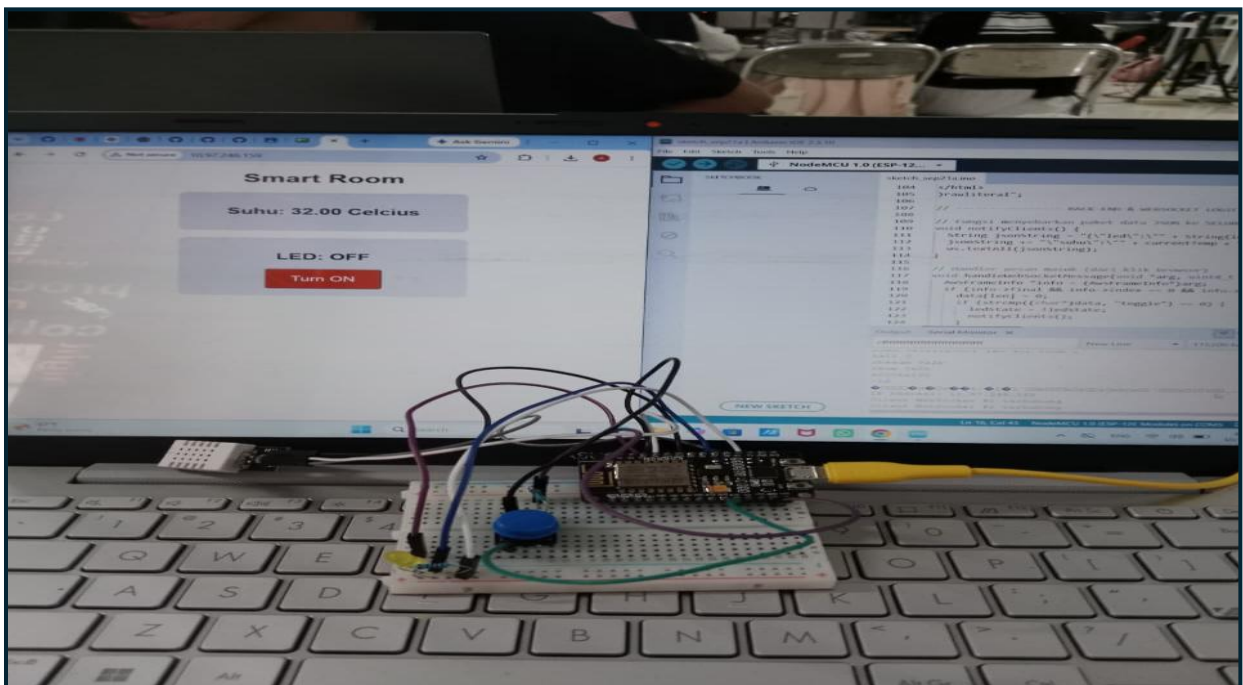
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK
NEGERI LAMPUNG
2026/2027

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik* SAAT ON

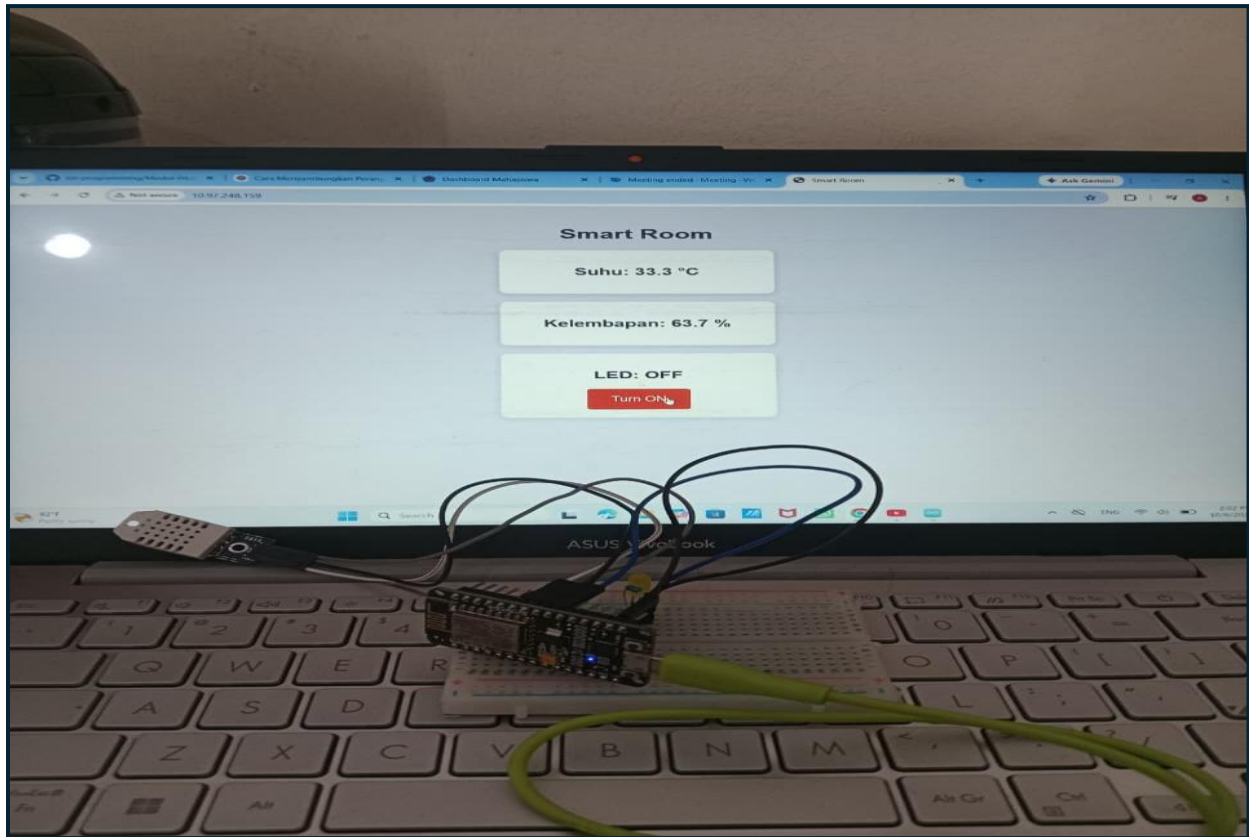


SAAT OFF

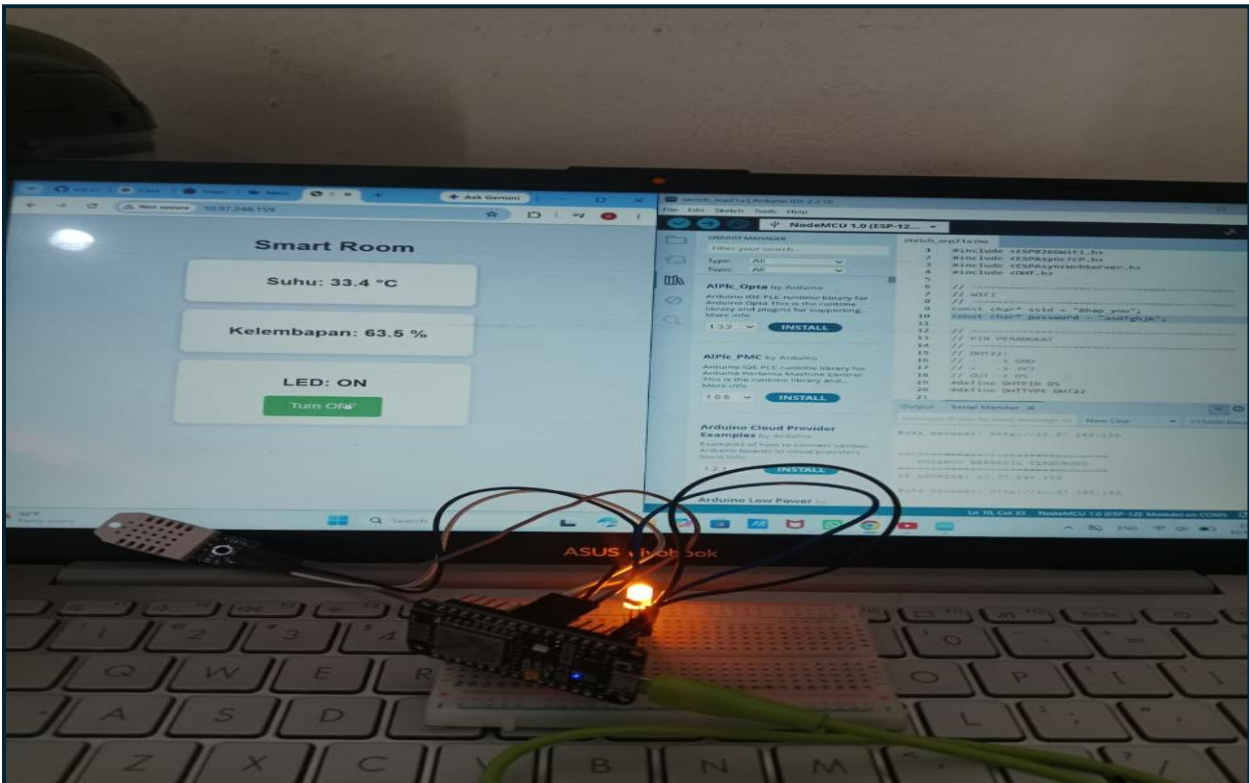


4.6 Latihan

SAAT OFF



SAAT ON



Latihan 1 (Teori Jaringan):

Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi WebSockets jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode Meta Refresh HTTP HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban:

WebSockets lebih hemat karena koneksi tetap terbuka sehingga data bisa dikirim langsung tanpa melakukan refresh halaman berulang kali. Sedangkan Meta Refresh harus memuat ulang halaman sehingga lebih banyak menggunakan bandwidth dan membuat layar terlihat berkedip.

Latihan 2

(Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets (reaksi antara browser dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawaban:

Jika menggunakan delay(3000), NodeMCU akan berhenti selama 3 detik sehingga komunikasi WebSockets menjadi lambat. Akibatnya, data dan perintah dari browser bisa terlambat diproses. Lebih baik menggunakan millis() agar proses tetap berjalan tanpa berhenti.

Latihan 3 (Modifikasi Front-End):

Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban:

ID HTML yang menjadi target JavaScript adalah ID pada elemen yang digunakan untuk menampilkan status. Contohnya:

ledStatus

ledButtonText

ID tersebut digunakan untuk mengubah teks status LED dan tombol pada halaman web.

Latihan 4

(Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban:

Kelembapan dapat ditambahkan dengan membaca `dht.readHumidity()` lalu mengirimkannya melalui JSON WebSockets.

C++:

```
String currentHum = "--";
```

```
float hum = dht.readHumidity();  
if (!isnan(hum)) {  
    currentHum = String(hum, 1);  
}
```

JSON:

```
json += "\"hum\": \"" + currentHum + "\"";
```

HTML:

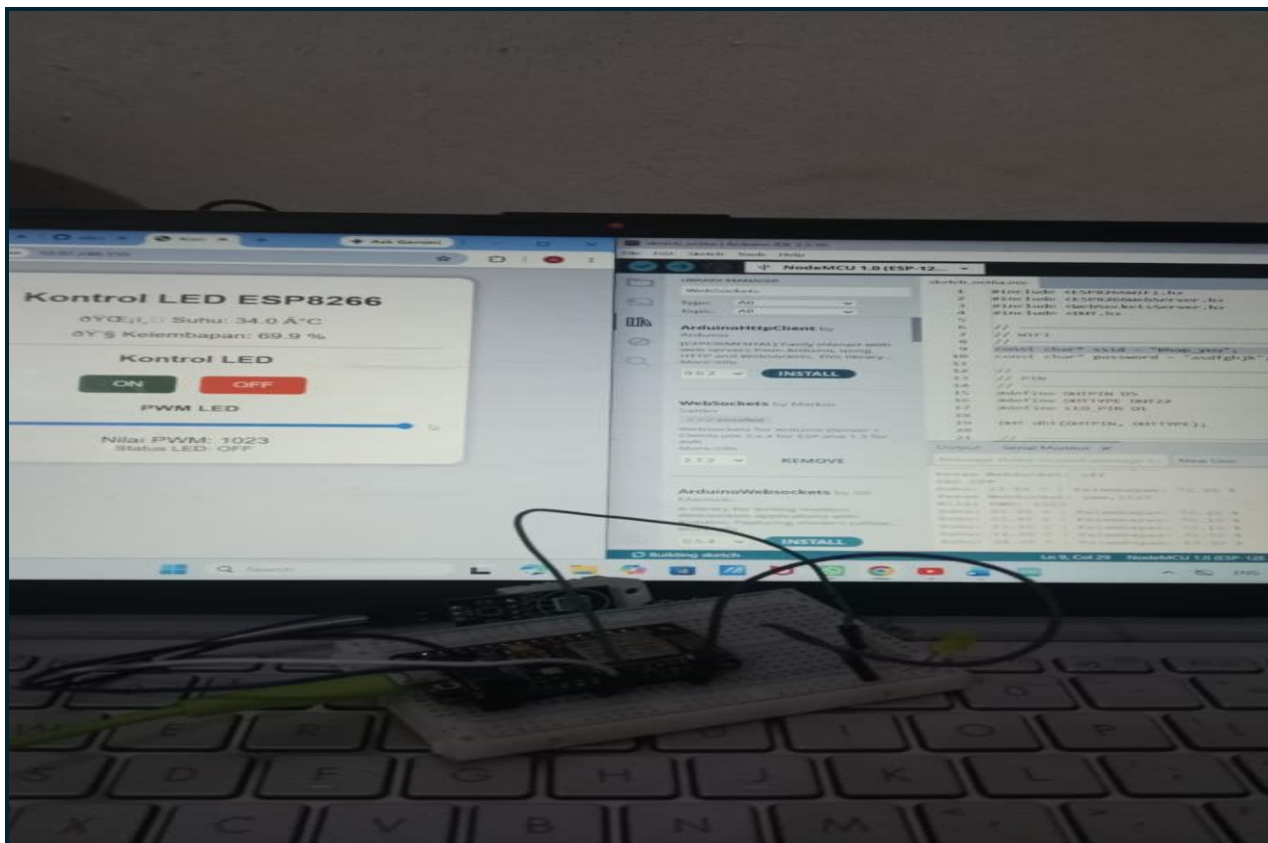
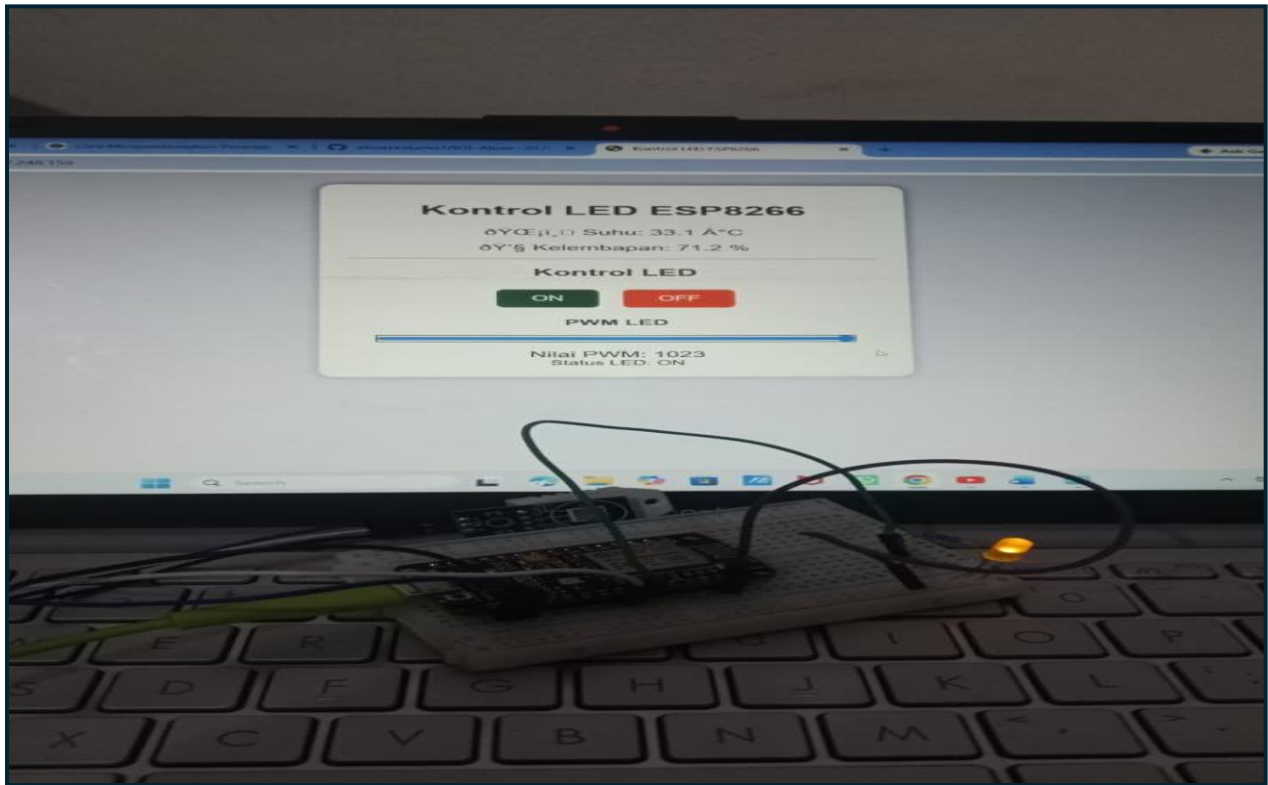
```
<h2>Kelembapan: <span id="humValue">--</span> %</h2>
```

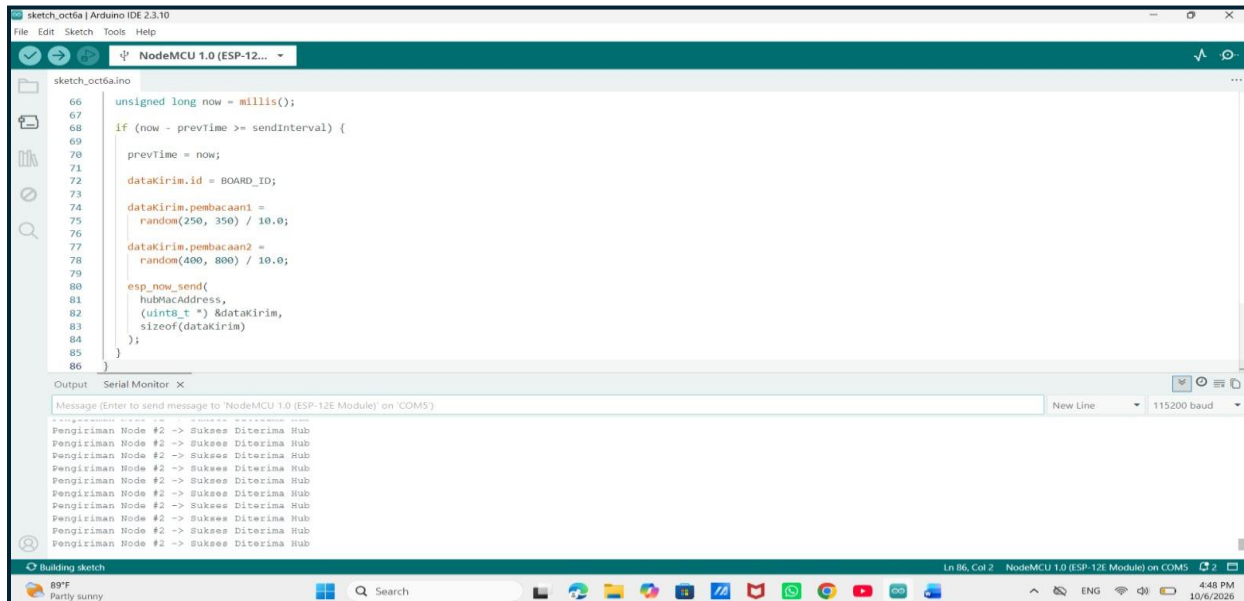
JavaScript:

```
document.getElementById("humValue").innerText = data.hum;
```

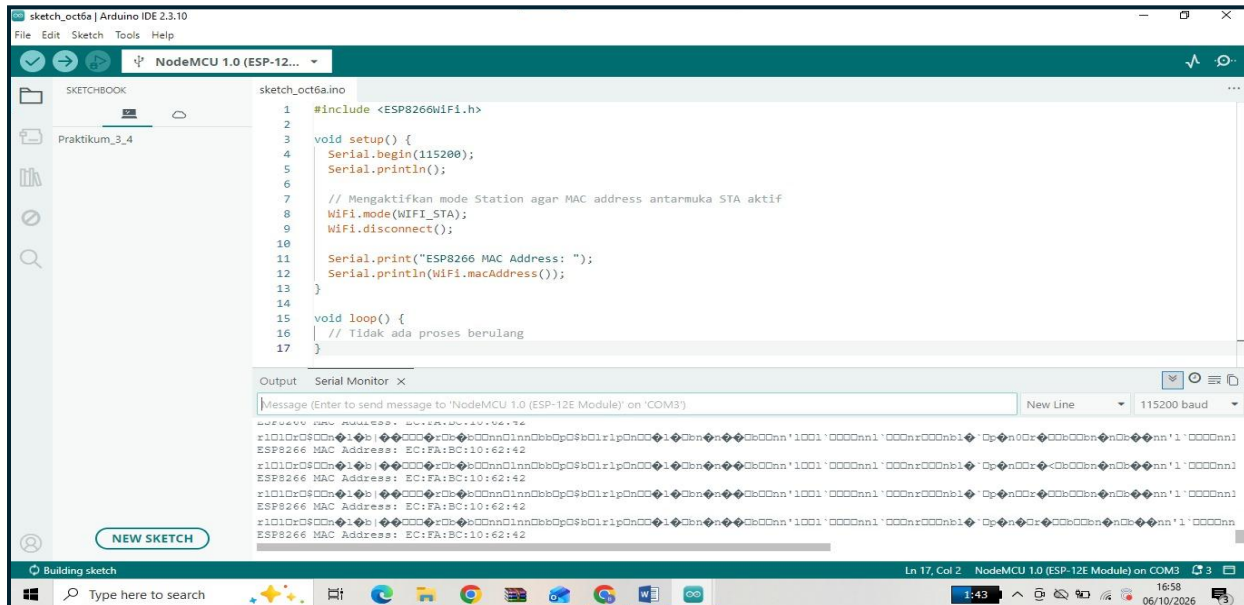
Dengan begitu, nilai kelembapan dari DHT22 bisa ditampilkan secara real-time di halaman web.

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via WebSockets





MAC Address 3



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Output Serial Monitor

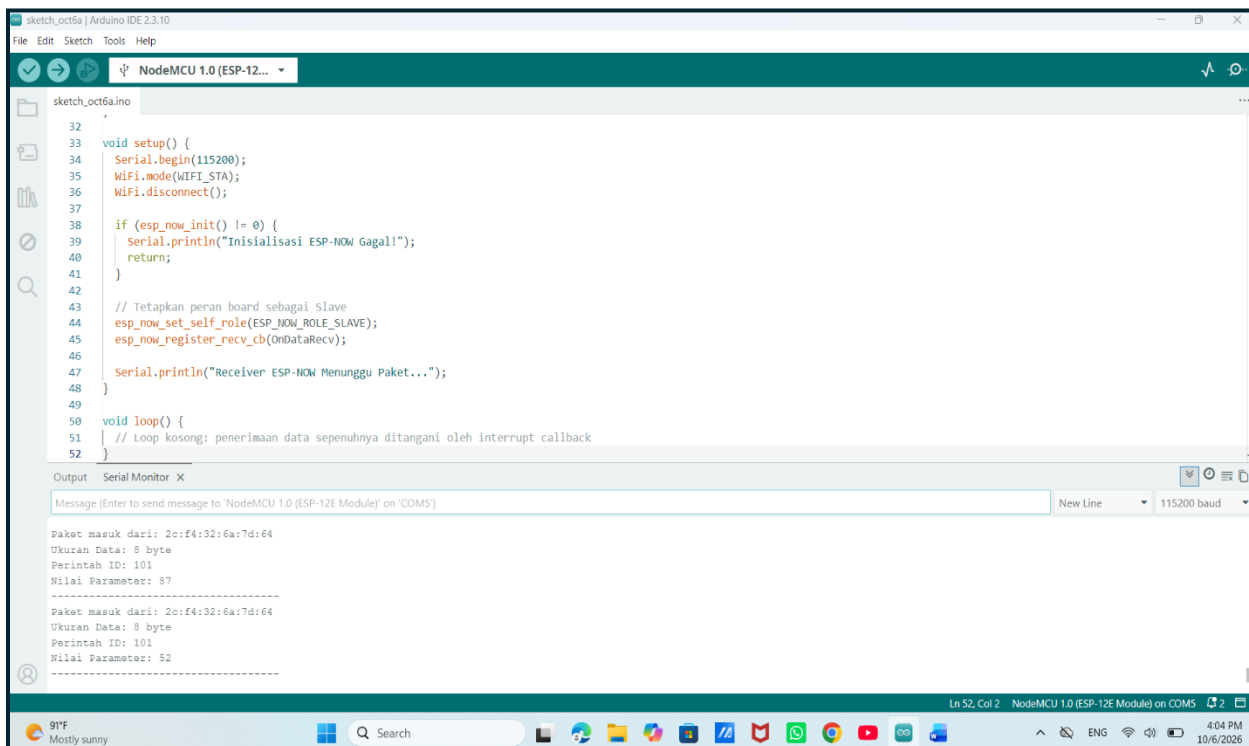
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

ESP8266 MAC Address: EC:FA:BC:10:62:42

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Laptop 1



```
32
33 void setup() {
34   Serial.begin(115200);
35   WiFi.mode(WIFI_STA);
36   WiFi.disconnect();
37
38   if (esp_now_init() != 0) {
39     Serial.println("Inisialisasi ESP-NOW Gagal!");
40     return;
41   }
42
43   // Tetapkan peran board sebagai Slave
44   esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
45   esp_now_register_recv_cb(OnDataRecv);
46
47   Serial.println("Receiver ESP-NOW Menunggu Paket...");
48 }
49
50 void loop() {
51   // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
52 }
```

Output Serial Monitor

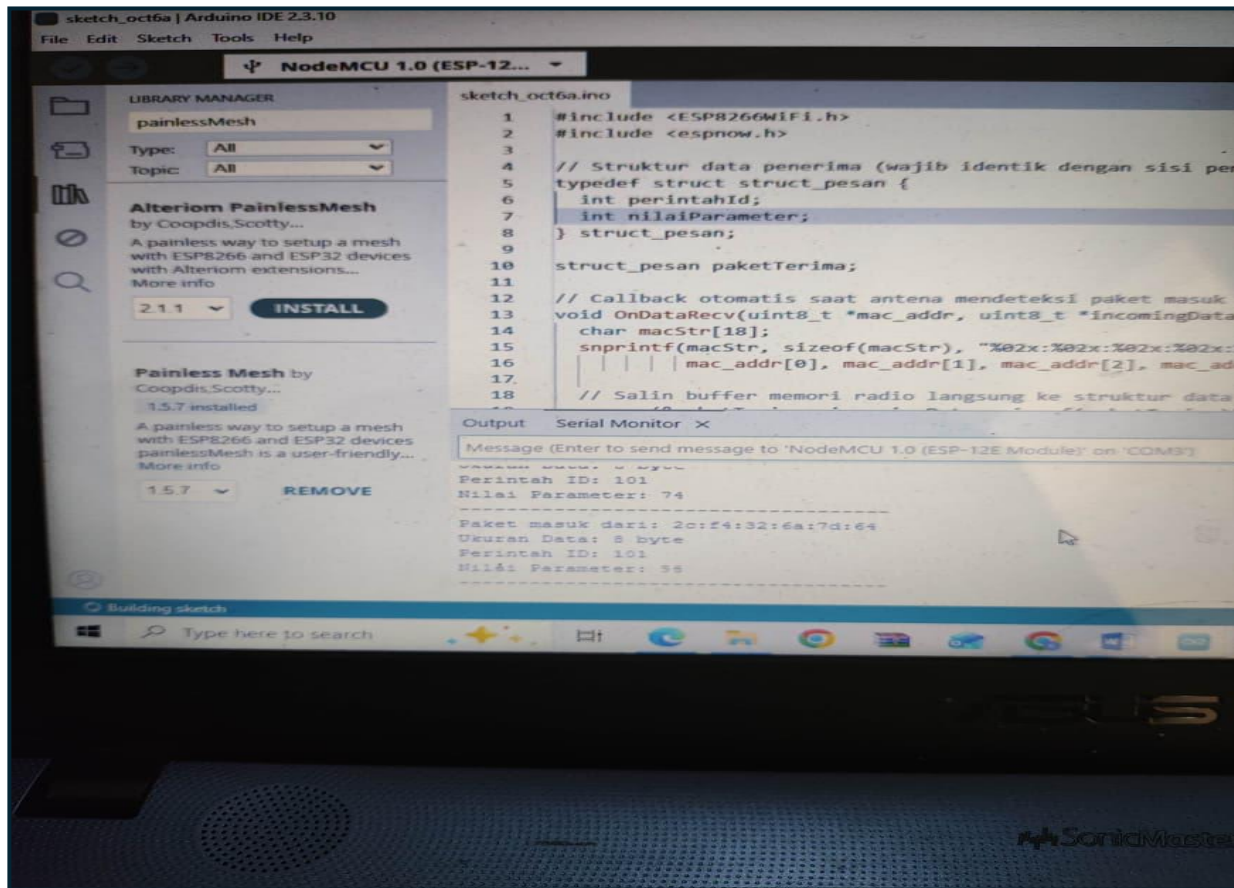
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

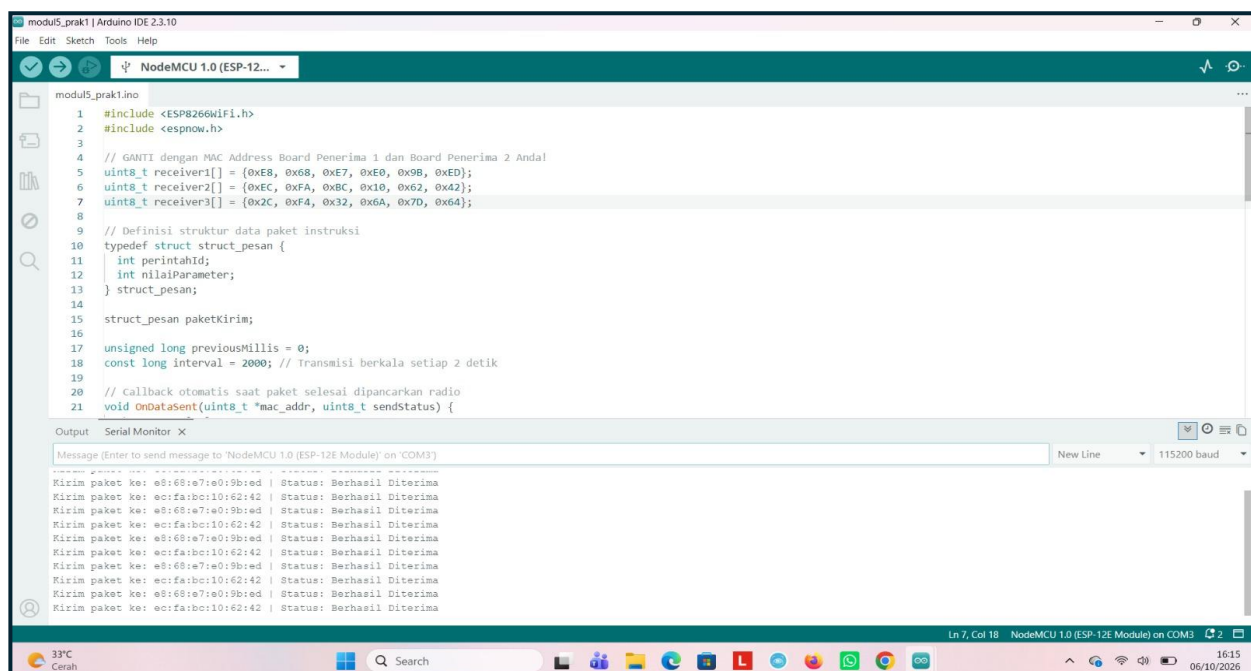
Paket masuk dari: 2c:f4:32:6a:7d:64
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 87

Paket masuk dari: 2c:f4:32:6a:7d:64
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 52

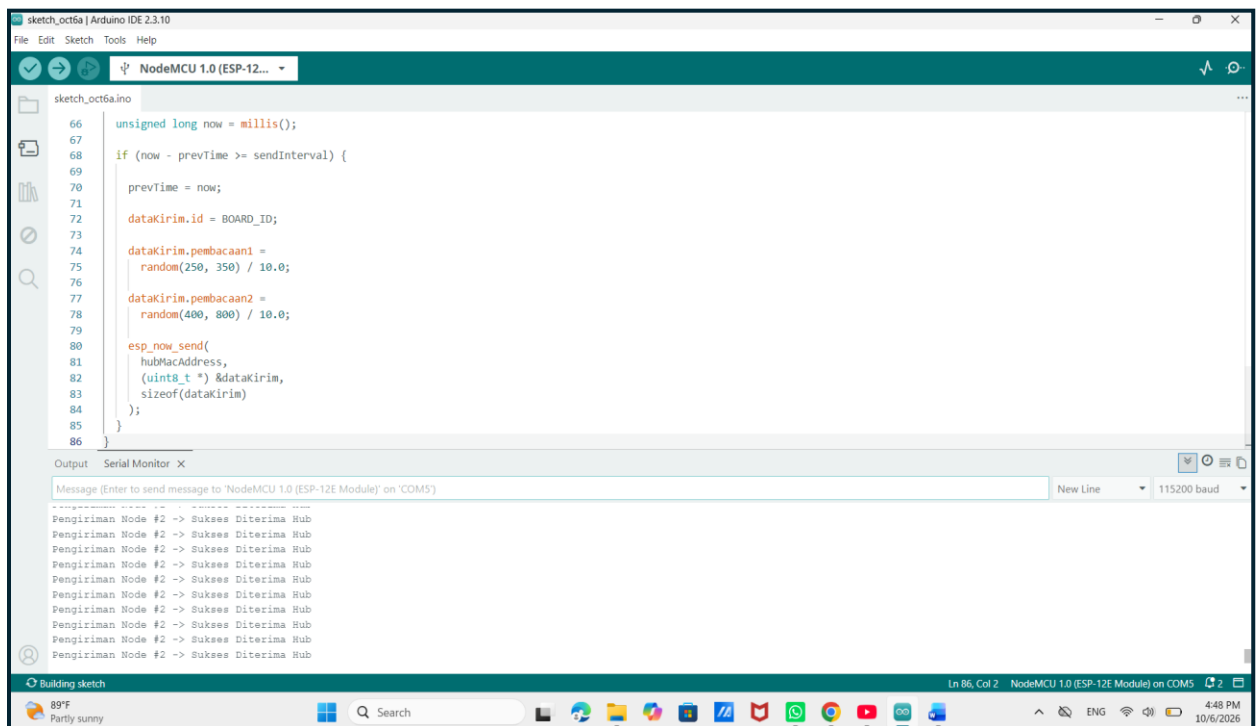
Laptop 2



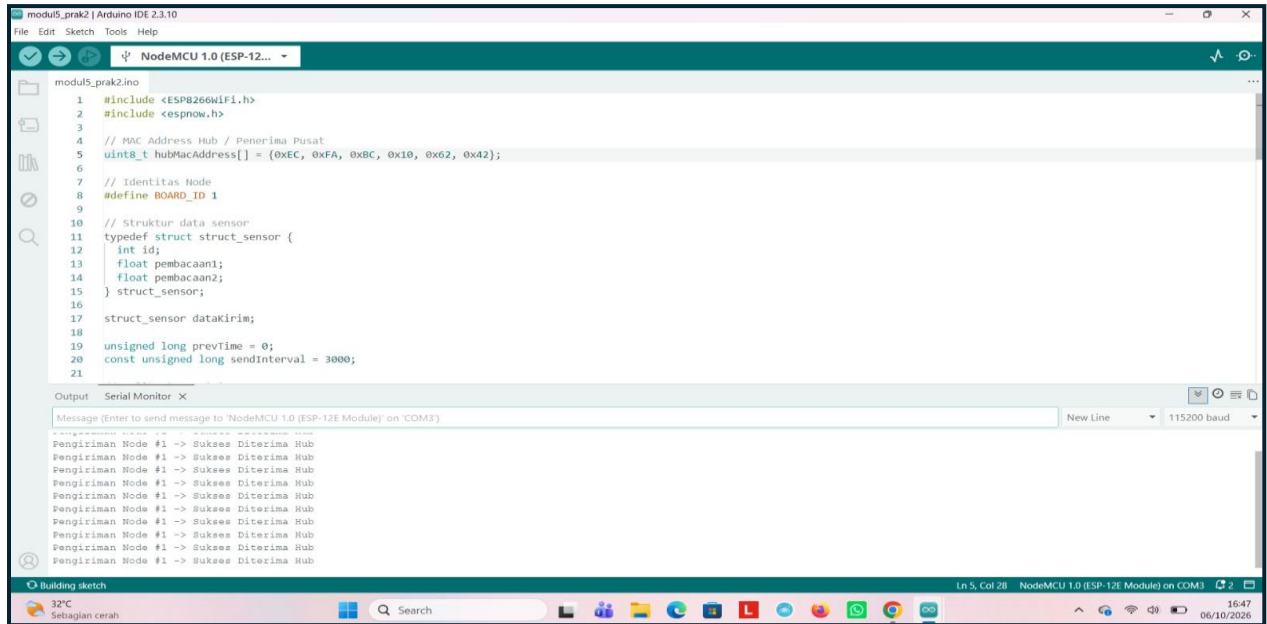
Laptop 3



Laptop 1



Laptop 3



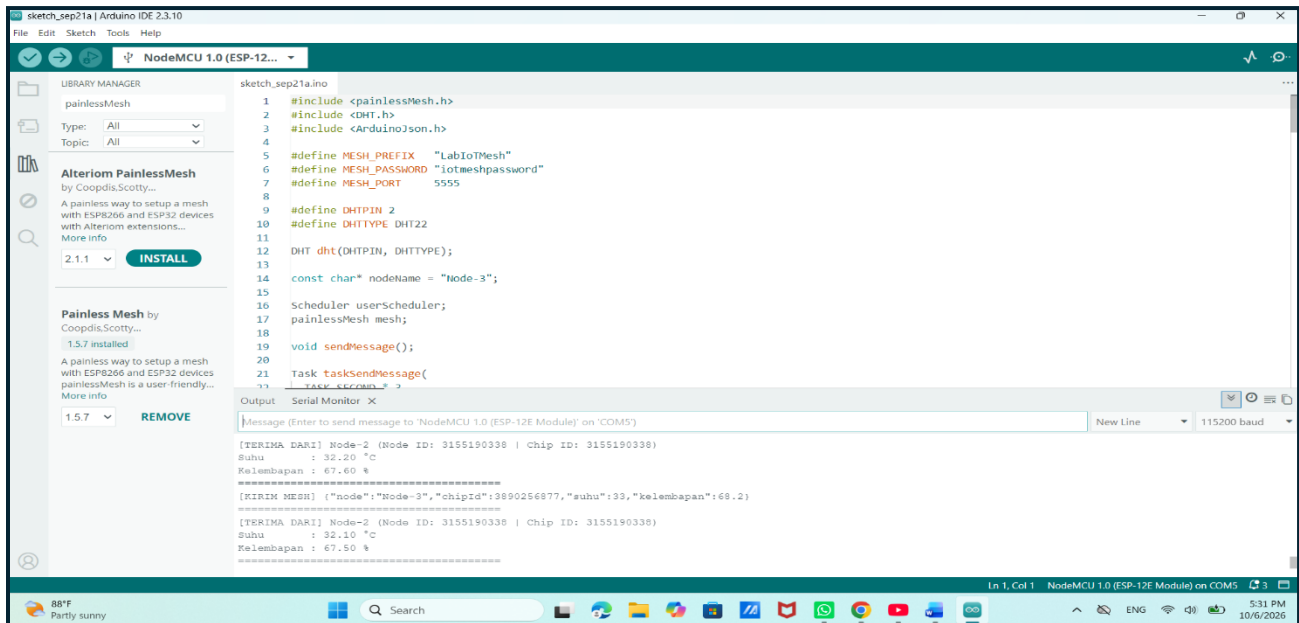
The screenshot shows the Arduino IDE interface with a sketch named 'modul5_prak2.ino' for a NodeMCU 1.0 (ESP-12E) module. The sketch includes headers for ESP8266 WiFi and ESPNow, defines a MAC address, a board ID, and a sensor structure. The serial monitor shows the output of the sketch, which is a series of 'Pengiriman Node #1 -> Sukses Diterima Hub' messages.

```
1 #include <ESP8266WiFi.h>
2 #include <ESPNow.h>
3
4 // MAC Address Hub / Penerima Pusat
5 uint8_t hubMacAddress[] = {0xEC, 0xFA, 0xBC, 0x10, 0x62, 0x42};
6
7 // Identitas Node
8 #define BOARD_ID 1
9
10 // Struktur data sensor
11 typedef struct struct_sensor {
12   int id;
13   float pembacaan1;
14   float pembacaan2;
15 } struct_sensor;
16
17 struct_sensor dataKirim;
18
19 unsigned long prevTime = 0;
20 const unsigned long sendInterval = 3000;
21
```

Serial Monitor Output:

```
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
```

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

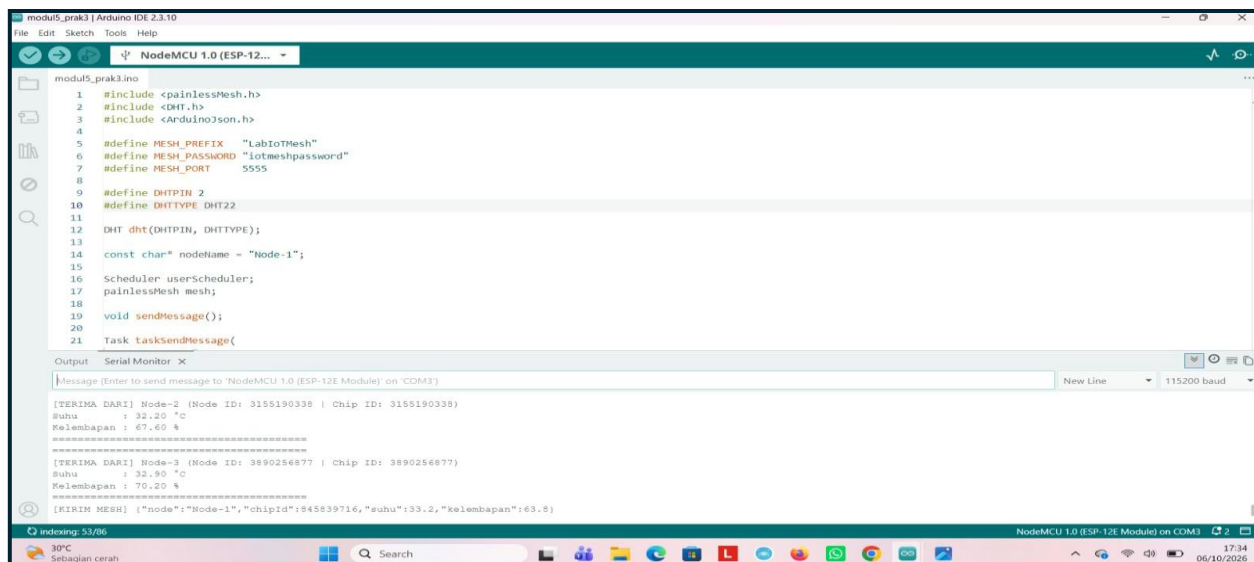
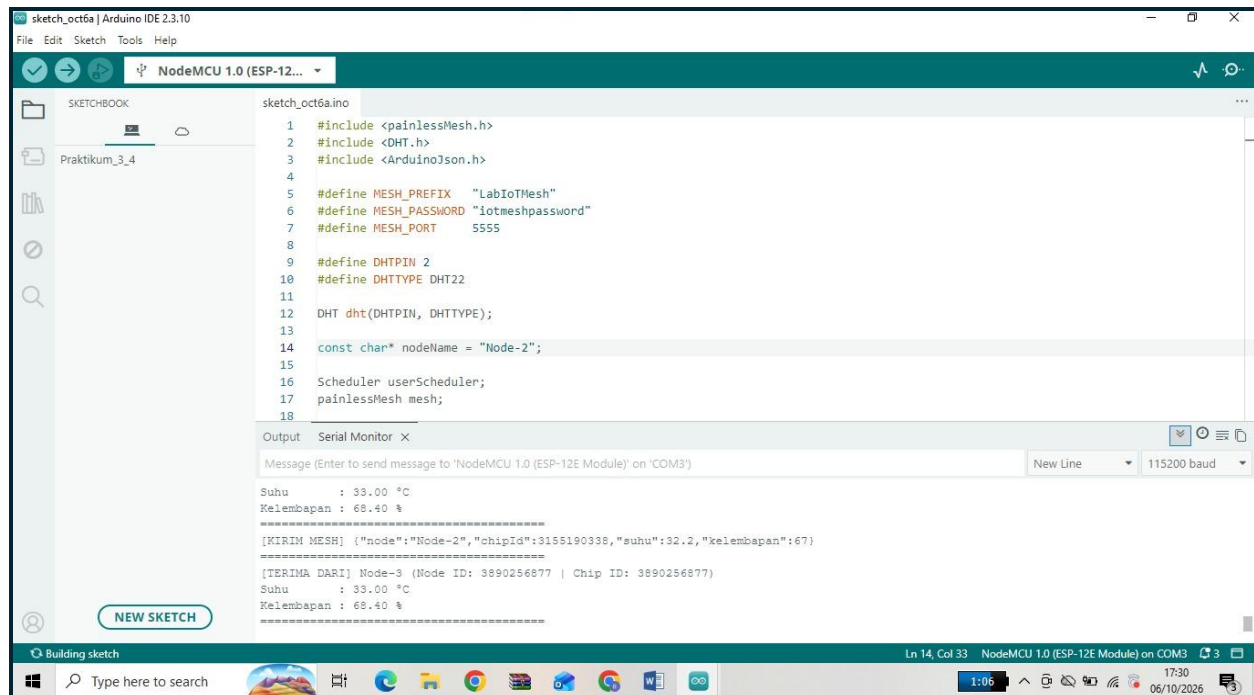


The screenshot shows the Arduino IDE interface with a sketch named 'sketch_sep21a.ino' for a NodeMCU 1.0 (ESP-12E) module. The sketch includes headers for painlessMesh, DHT, and ArduinoJson. It defines mesh prefix, password, port, DHT pin, and type. The serial monitor shows the output of the sketch, which is a series of 'TERIMA DARI' and 'KIRIM MESH' messages.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "Iotmeshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 const char* nodeName = "Node-3";
15
16 Scheduler usersScheduler;
17 painlessMesh mesh;
18
19 void sendMessage();
20
21 Task taskSendMessage(
22   TASK_SECOND * 2
23 )
```

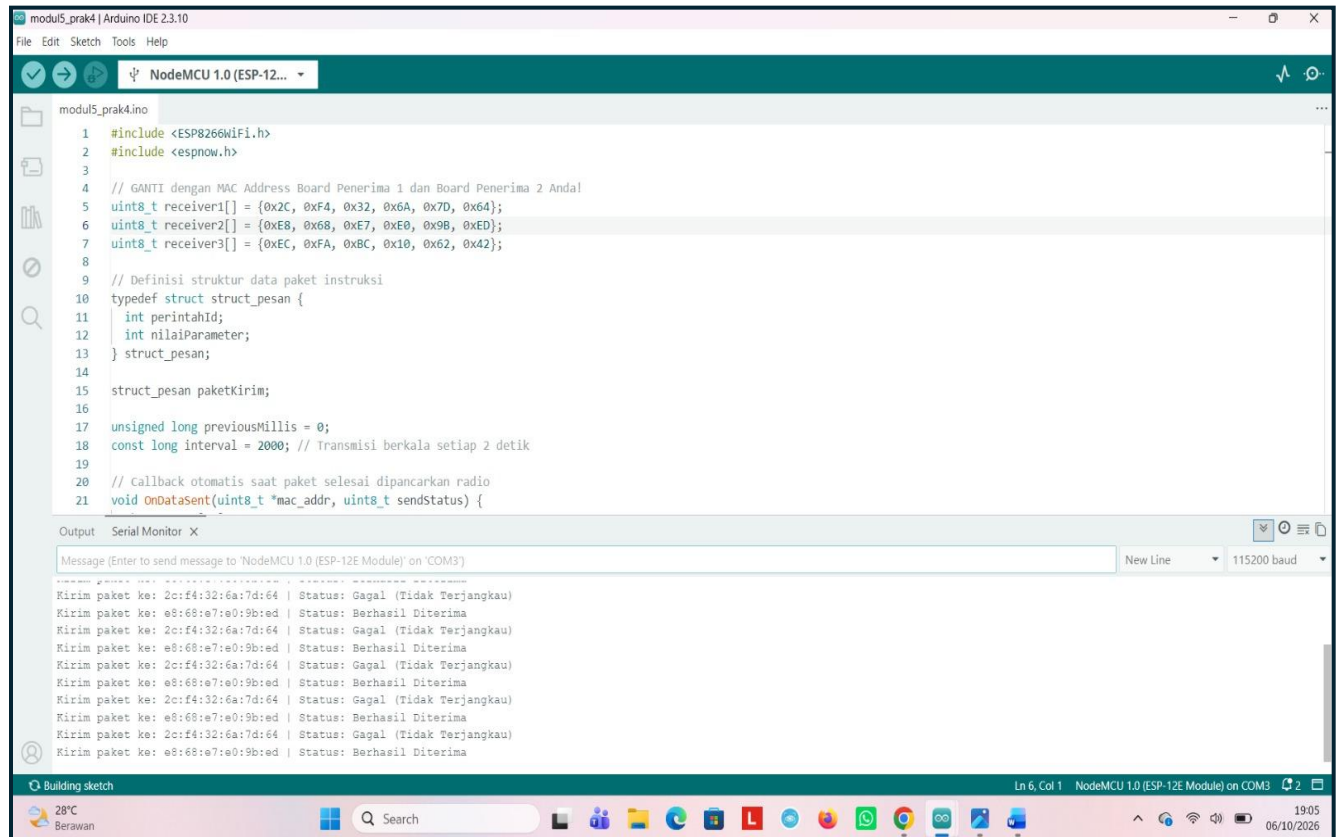
Serial Monitor Output:

```
[TERIMA DARI] Node-2 (Node ID: 3155190338 | Chip ID: 3155190338)
Suhu : 32.20 °C
Kelembapan : 67.60 %
=====
[KIRIM MESH] ("node": "Node-3", "chipId": 3890256877, "suhu": 33, "kelembapan": 68.2)
=====
[TERIMA DARI] Node-2 (Node ID: 3155190338 | Chip ID: 3155190338)
Suhu : 32.10 °C
Kelembapan : 67.50 %
```



5.6 Latihan

Latihan 1



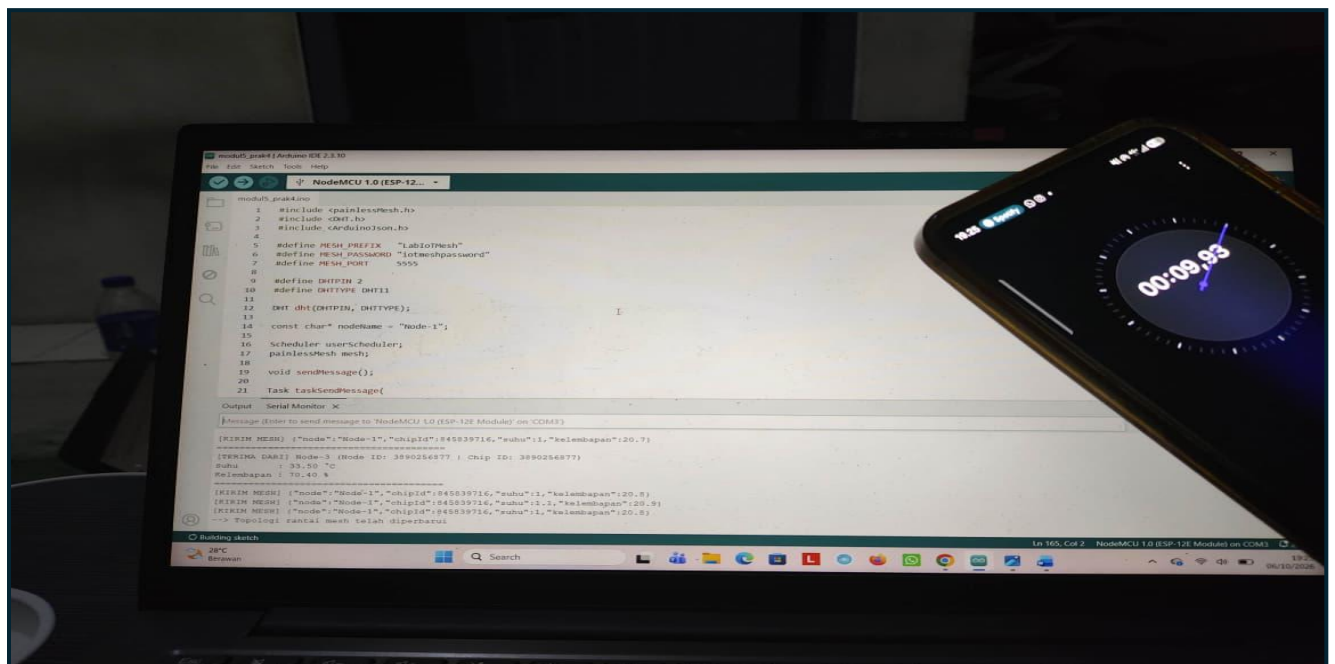
The screenshot shows the Arduino IDE interface with a sketch named 'modul5_prak4.ino' for a NodeMCU 1.0 (ESP-12E) module. The code defines MAC addresses for three boards, a packet structure, and a function to send packets. The Serial Monitor shows the output of the program, indicating successful and failed transmissions.

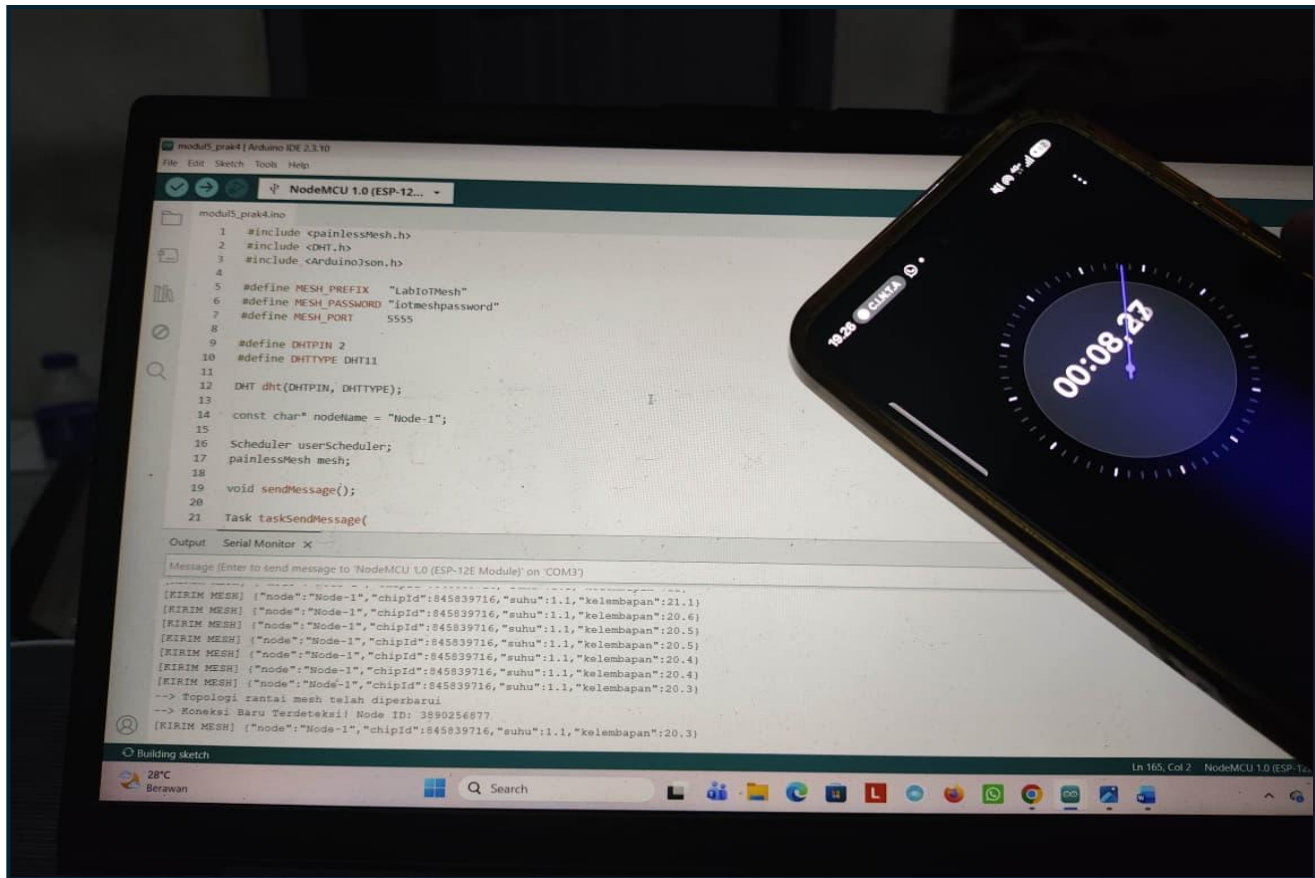
```
modul5_prak4.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andar
5 uint8_t receiver1[] = {0x2C, 0xF4, 0x32, 0x6A, 0x7D, 0x64};
6 uint8_t receiver2[] = {0xE8, 0x68, 0xE7, 0xE0, 0x9B, 0xED};
7 uint8_t receiver3[] = {0xEC, 0xFA, 0x8C, 0x10, 0x62, 0x42};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 // Callback otomatis saat paket selesai dipancarkan radio
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

Serial Monitor Output:

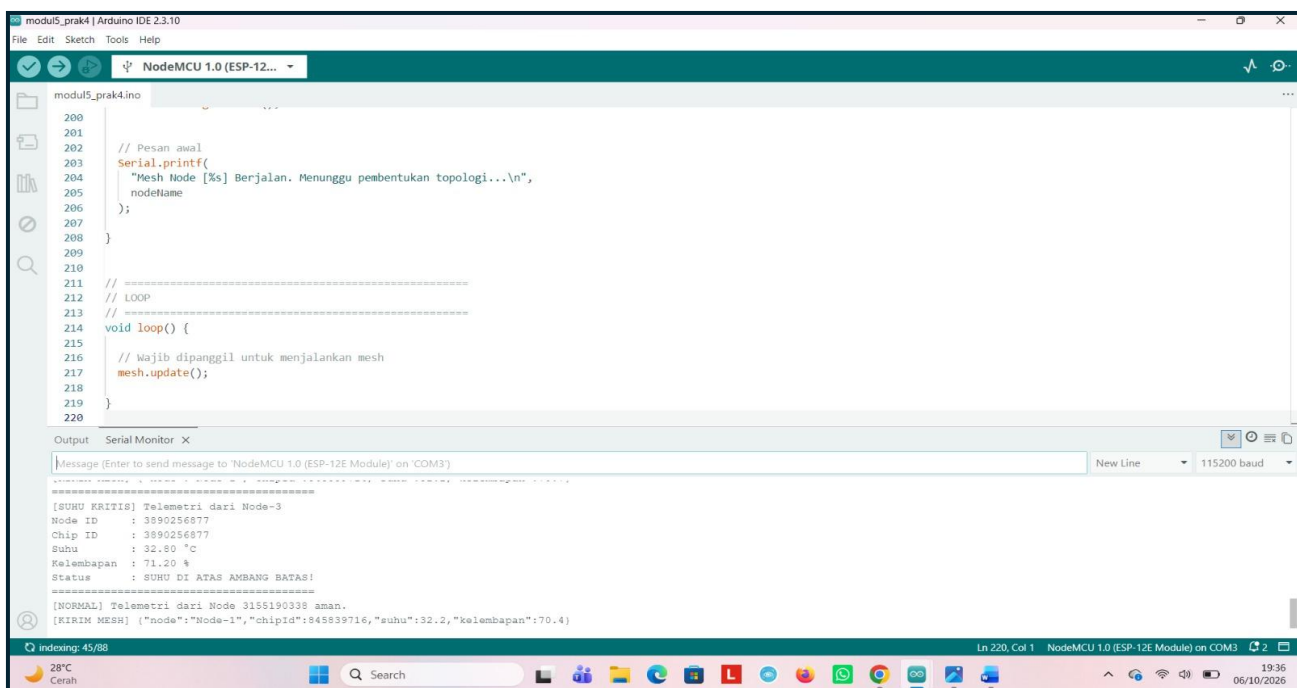
```
=====
Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima
```

Latihan 2





Latihan 3



(Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

- i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawaban:

Saat NodeMCU 2 dicabut, Controller akan mencoba mengirim data ke MAC NodeMCU 2 tetapi tidak mendapat balasan ACK. Akibatnya status pengiriman berubah menjadi:

Gagal (Tidak Terjangkau)

ESP-NOW tetap bisa mengetahui pengiriman gagal karena bekerja pada **Layer 2 (MAC)** dan memiliki mekanisme konfirmasi penerimaan paket. Jadi tidak membutuhkan koneksi TCP untuk mengetahui apakah perangkat tujuan masih bisa menerima data.

Latihan 2

(Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4): Jalankan jaringan *mesh* pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai *mesh*?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi *mesh* tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawaban:

Saat salah satu Node dicabut atau di-reset, node lain akan mendeteksi perubahan koneksi melalui `changedConnectionCallback()`. Waktu deteksi dicatat berdasarkan hasil pengamatan pada Serial Monitor.

Contoh hasil pengamatan:

```
Waktu node terputus : ± beberapa detik  
Waktu node terhubung : ± beberapa detik
```

Setelah Node dinyalakan kembali, `painlessMesh` dapat mencari dan terhubung kembali ke node lain secara otomatis tanpa melakukan konfigurasi ulang.

Latihan 3**(Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):**

Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal ($\leq 32.0\text{ °C}$), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetry dari Node <chipId> aman`.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawaban:

```
```cpp
```

```
// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
```

```
void receivedCallback(uint32_t from, String &msg) {
```

```
 // Parsing muatan teks JSON yang diterima
```

```
 StaticJsonDocument<200> doc;
```

```
DeserializationError error = deserializeJson(doc, msg);
```

```
if (!error) {
```

```
 const char* sender = doc["node"];
```

```
 uint32_t chipId = doc["chipId"];
```

```
 float suhu = doc["suhu"];
```

```
 float kelembapan = doc["kelembapan"];
```

```
 // =====
```

```
 // FILTER SUHU KRITIS
```

```
 // Hanya tampilkan telemetry lengkap jika suhu > 32.0 °C
```

```
 // =====
```

```
 if (suhu > 32.0) {
```

```
 Serial.println("=====");
```

```
 Serial.printf("[SUHU KRITIS] Telemetry dari %s\n", sender);
```

```
 Serial.printf("Node ID : %u\n", from);
```

```
 Serial.printf("Chip ID : %u\n", chipId);
```

```
 Serial.printf("Suhu : %.2f °C\n", suhu);
```

```
 Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
```

```
 Serial.println("Status : SUHU DI ATAS AMBANG BATAS!");
```

```
 Serial.println("=====");
```

```
 } else {
```

```
 // Jika suhu normal, hanya tampilkan satu baris
```

```
Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);

}

}else{

// Jika data yang diterima bukan JSON yang valid
Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());

}

}

` ``
```

## **5.7 Tugas: Sistem Telemetry & Otomasi Terdistribusi Berbasis painlessMesh**